SDLC and the DevOps Lifecycle: Understanding How Software Reaches Production

When people start learning DevOps, it is easy to focus immediately on tools such as Docker, Kubernetes, Terraform, Jenkins, GitHub Actions, or AWS. The problem with learning this way is that the tools can feel disconnected. You may understand what a command does without understanding why the tool exists or where it fits into the software delivery process.

A better starting point is the Software Development Life Cycle, or SDLC.

The SDLC describes how software moves from an initial idea to a production system that is used, monitored, maintained, and improved over time. DevOps does not replace the SDLC. Instead, it changes how teams execute the lifecycle by introducing stronger collaboration, automation, continuous delivery practices, infrastructure management, and operational feedback.

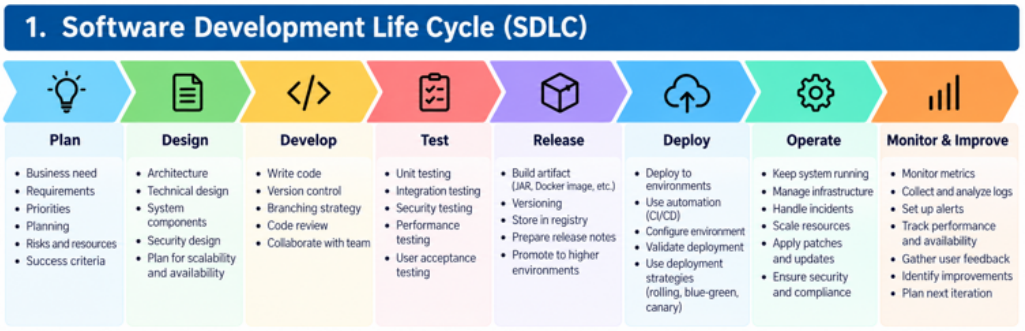
Understanding this relationship provides the context needed to make sense of the rest of the DevOps toolchain.

Understanding the Software Development Life Cycle

The Software Development Life Cycle is the structured process used to design, build, test, release, operate, and maintain software.

Although different organizations may use different methodologies, most software passes through the same general activities: requirements are identified, a solution is designed, developers implement the software, the system is tested, a release is prepared, the application is deployed, and the production environment is monitored and maintained.

The important point is that software development does not end when the code is written. A feature that exists on a developer's laptop has no business value until it can be tested, delivered safely, operated reliably, and supported in production.



Consider a simple e-commerce example. A company wants customers to receive an email when an order ships.

The requirement itself is straightforward, but implementing it involves more than adding a few lines of code. The team must determine what event triggers the notification, how the

application communicates with the email service, what happens if the email provider is unavailable, how failures are logged, how the feature is tested, how the change is deployed, and how the team will know if notifications stop working.

This single feature touches multiple parts of the SDLC.

Planning and Requirements

The lifecycle begins with a business need.

In the shipping-notification example, the business requirement may be expressed as:

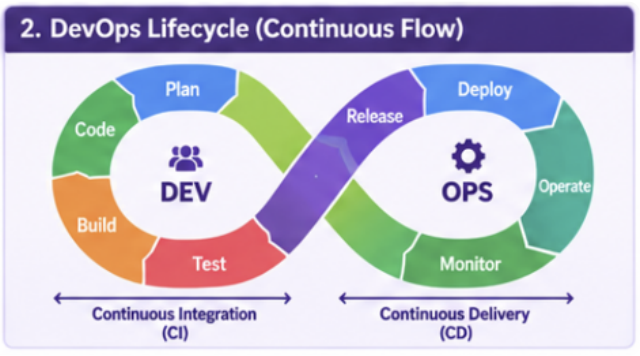
> Customers should receive a shipping confirmation email after their order is dispatched.

The team then converts the business request into technical and functional requirements. Acceptance criteria may specify that the email contains the order number and tracking information, failed notifications must be logged, and failed deliveries should be retried.

This stage is important because unclear requirements create problems later. A technically correct implementation can still fail if the team misunderstood what the business expected.

Tools such as **Jira, Confluence, Azure Boards, or GitHub Issues** are often used to record requirements, decisions, tasks, and acceptance criteria.

DevOps also begins to influence the project at this stage. Operational questions should be raised before the application reaches production. For example, the team may need to determine whether the feature requires a new service, additional cloud infrastructure, new secrets, a message queue, monitoring, or changes to the deployment pipeline. This is one of the fundamental ideas behind DevOps: operational concerns should be considered throughout the lifecycle rather than only after development is complete.



Design and Architecture

Once the requirements are understood, the team determines how the solution should be implemented.

For the notification feature, one possible architecture could involve the order service publishing an event when an order is shipped. A notification service consumes the event and

communicates with an external email provider.



At this stage, engineers make decisions about application components, APIs, databases, service communication, authentication, networking, security, scalability, and failure handling.

From a DevOps perspective, architecture also includes operational concerns.

Where will the service run? How will it communicate with other services? How will configuration be provided? How will credentials be protected? How will the service be deployed? What metrics will indicate whether it is healthy?

These questions connect software architecture with production architecture.

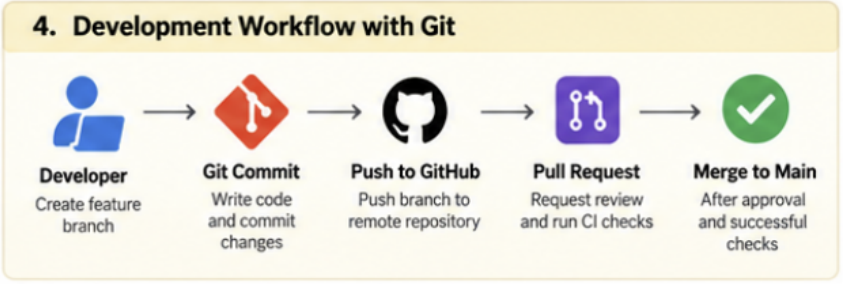
Development and Version Control

Once the design is agreed upon, developers begin implementation.

Source code is normally maintained using a version control system such as Git. Developers work in branches, commit their changes, push them to a shared repository, and create pull requests for review.

A typical development workflow may include creating a feature branch, implementing the change, running local tests, pushing the branch, and requesting review before merging the change into the main codebase.

Version control provides more than source-code storage. It records the history of the application, enables collaboration, supports code review, and often acts as the entry point to automated delivery pipelines. In modern development environments, a push or pull request can automatically trigger testing, security checks, builds, and other validation steps.

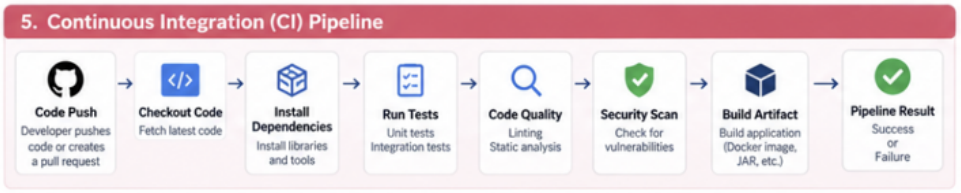


This is where development begins to connect directly with DevOps automation.

Continuous Integration

When multiple developers contribute to the same codebase, changes must be validated before they are merged or released. Continuous Integration, commonly abbreviated as CI, is the practice of integrating changes frequently and validating them automatically.

A CI pipeline may install dependencies, compile the application, execute unit tests, run static analysis, perform security checks, and build a deployable artifact. If one of these steps fails, the pipeline reports the failure and prevents the change from moving forward.



The objective is not simply automation for its own sake. CI reduces the risk of discovering integration problems late in the delivery process. Instead of waiting until release time to find out whether several changes work together, the system validates changes continuously.

Common CI platforms include **GitHub Actions, GitLab CI/CD, Jenkins, Azure DevOps, and CircleCI**.

Build and Artifact Creation

Once the code has passed the required validation, the application must be packaged into something that can be deployed.

The exact artifact depends on the technology. A Java application may produce a JAR file, a frontend application may produce static assets, and a containerized application may produce a Docker image.

In container-based environments, the application and its runtime dependencies are packaged into an image. The image can then be executed consistently across different environments.

For example, the notification service may be packaged as:

`notification-service:1.0.0`

The important idea is that the deployment process should use a known, versioned artifact rather than rebuilding the application differently in every environment. This improves consistency and makes releases easier to trace.

Testing as Part of Delivery

Testing is not a single activity performed just before production. It happens at multiple levels throughout the lifecycle.

Unit tests validate individual functions or components. Integration tests validate communication between components. API tests verify service interfaces. End-to-end tests

validate complete workflows from the user's perspective.

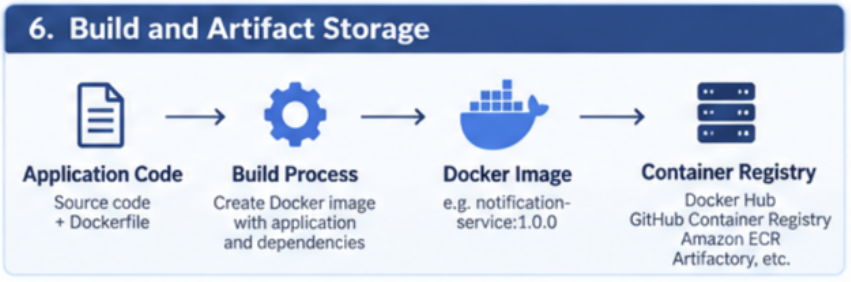
For the shipping-notification feature, an end-to-end test might verify that updating an order to a shipped state results in a notification being generated successfully. Security, performance, load, and regression testing may also be added depending on the system and its risk profile. In mature DevOps environments, many of these checks are integrated directly into the delivery pipeline. The goal is to identify problems as early as possible, when they are usually cheaper and easier to fix.

Artifact Repositories

After the application has been built and tested, the resulting artifact needs to be stored somewhere reliable.

Containerized applications are typically stored in a container registry such as **Amazon ECR, GitHub Container Registry, Docker Hub, or Google Artifact Registry**. Other types of packages may be stored in systems such as JFrog Artifactory or Sonatype Nexus.

The repository becomes the source of deployable application versions.



A common release principle is to build an artifact once and promote the same version through different environments such as development, testing, staging, and production.

This avoids subtle differences caused by rebuilding the application for every environment.

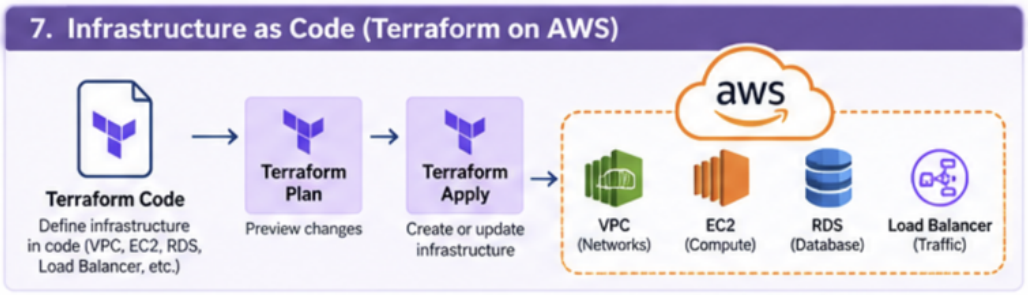
Infrastructure and Infrastructure as Code

A deployable application still needs infrastructure on which to run. Infrastructure may include **compute instances, networks, load balancers, databases, storage, DNS records, Kubernetes clusters, security groups, and identity permissions**.

Historically, many of these resources were created manually. An engineer might log into a cloud console, create servers, configure networks, and change settings by hand. This approach becomes difficult to manage as systems grow.

Manual configuration is hard to reproduce, hard to review, and easy to change inconsistently.

Infrastructure as Code addresses this problem by defining infrastructure using configuration files or code. Tools such as **Terraform** allow infrastructure to be version-controlled, reviewed, automated, and recreated.



This is a major DevOps principle because infrastructure becomes part of the software delivery process rather than a separate manual activity.

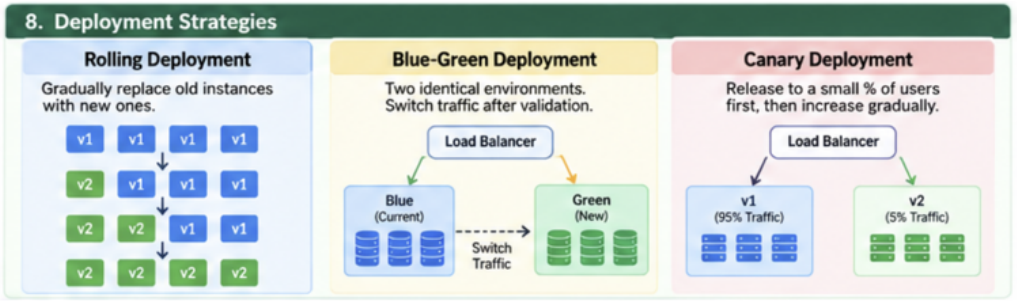
Deployment

Deployment is the process of taking a tested application artifact and running it in a target environment. The target may be a virtual machine, a container platform, a managed cloud service, or a Kubernetes cluster. Deployment can be performed manually, but mature environments usually automate the process.

CI/CD platforms and deployment tools may retrieve an artifact from a registry, update application configuration, deploy the new version, verify application health, and rollback if the deployment fails.

Different deployment strategies exist depending on the risk and availability requirements of the system.

A **rolling deployment** gradually replaces instances of the previous version. A **blue-green deployment** maintains two environments and switches traffic to the new environment after validation. A **canary deployment** sends a small percentage of production traffic to the new version before increasing exposure.



The objective is to reduce the risk associated with releasing new software.

Operations

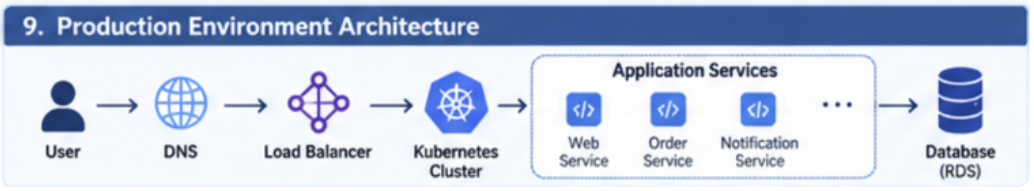
Once software is running in production, the focus changes from delivering the application to keeping it reliable. Production systems can fail for many reasons that have little to do with the application code itself.

A server may run out of disk space. A certificate may expire. DNS may be misconfigured. A container may repeatedly restart. A database connection may time out. A firewall rule may

block traffic. CPU or memory usage may increase unexpectedly. This is why DevOps engineers need strong foundations in operating systems, Linux, networking, cloud infrastructure, containers, security, and troubleshooting.

Suppose users report that an application is unavailable.

The issue could be at any point between the user and the application.



Troubleshooting requires following that path systematically.

Is DNS resolving? Is the destination reachable? Is the correct port open? Is the load balancer healthy? Is the service running? Is the container healthy? Is the application listening on the expected port? Can it connect to its dependencies?

This is where DevOps becomes less about individual tools and more about understanding systems.

Monitoring and Observability

A production system should not rely on users to report that something is broken. Teams need visibility into the health and behaviour of their applications and infrastructure.

Monitoring may track CPU usage, memory consumption, disk space, application availability, request latency, error rates, throughput, and container health.

Tools such as **Prometheus, Grafana, AWS CloudWatch, Datadog, and New Relic** are commonly used for this purpose.



Metrics answer questions such as:

- Is the application healthy?
- Is latency increasing?
- Are error rates rising?
- Are resources becoming saturated?

Logs provide another source of information.

A metric may show that error rates increased, while application logs may reveal that the underlying cause is a database timeout or authentication failure.

Centralized logging tools such as **Elastic Stack, Loki, Splunk, or CloudWatch Logs** allow engineers to investigate systems without manually logging into each server.

Monitoring becomes more effective when combined with alerting. Instead of watching dashboards continuously, teams can configure thresholds and conditions that notify engineers when intervention may be required.

Feedback Closes the Lifecycle

The production environment generates information that should influence future development.

For example, monitoring might reveal that checkout latency increases significantly during periods of high traffic. That observation may become a new engineering task. The team investigates the issue, plans an improvement, changes the implementation, validates the change, and deploys a new version.

The lifecycle therefore does not end at deployment. Production feedback becomes input for the next development cycle.

This continuous feedback loop shown in image 2 is one of the defining characteristics of modern DevOps.

SDLC and DevOps Are Related, but They Are Not the Same

The SDLC describes the overall lifecycle through which software is created and maintained.

DevOps is a set of practices, cultural principles, and technical approaches used to improve how teams execute that lifecycle.

The SDLC answers the question:

> ℹ What stages does software go through?

DevOps adds another question:

> ℹ How can we make those stages faster, safer, more repeatable, more collaborative, and more observable?

That distinction is important. DevOps is not another stage added after development. It influences the entire delivery process.

Where the Common DevOps Tools Fit

Once the lifecycle is clear, individual tools become easier to understand.

- Git provides version control and collaboration.
- GitHub Actions or Jenkins automate integration and delivery workflows.
- Docker provides a consistent way to package applications.
- Artifact registries store versioned application packages.

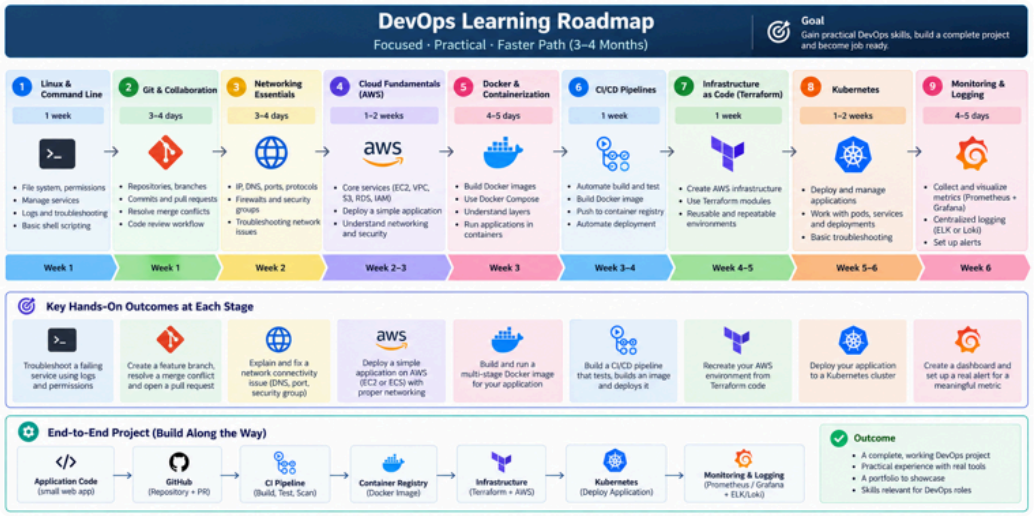
- Terraform automates infrastructure creation.
- AWS, Azure, or Google Cloud provide the infrastructure on which applications run.
- Kubernetes manages containerized workloads at scale.
- Prometheus collects metrics.
- Grafana visualizes operational data.
- Logging platforms help investigate application and infrastructure behaviour.

Each tool exists because it solves a specific problem within the lifecycle.

This is a much more useful way to learn DevOps than memorizing a list of technologies.

My DevOps Learning Roadmap

Because I already have a software development and technical background, I am approaching DevOps with a focused, hands-on roadmap rather than treating every topic as a long standalone course. My goal is to build enough depth to understand how each part of the delivery lifecycle works in practice: Linux and system troubleshooting, Git and collaboration, networking, cloud fundamentals, containerization, CI/CD, Infrastructure as Code, Kubernetes, and monitoring. I will learn each stage by applying it to the same project so that the individual tools gradually form one complete system. This should help me move beyond isolated tutorials and develop the ability to trace how code travels from a developer machine to production, how infrastructure supports it, how deployments are automated, and how problems are detected and resolved after release.



Conclusion

The most useful idea I took away from studying the SDLC is that software delivery is a connected system. Code is only one part of it. A production application depends on requirements, architecture, version control, testing, build systems, infrastructure, deployment, networking, monitoring, security, and operational processes. DevOps connects

these activities and aims to make the entire path from development to production more reliable and repeatable.

Understanding this lifecycle gives context to every DevOps technology that comes afterward.